

Estudo de Python e dados III

Manipulação de dados: Tipos de dados, Strings e dados do tipo texto, Apply, Operações groupby: separar-aplicar-combinar e O tipo de dado datetime.

Sergio Pedro Rodrigues Oliveira

05 junho 2026

SUMÁRIO

1	Objetivo	1
2	Tipos de dados	2
2.1	Introdução	2
	Objetivos	2
2.2	Tipos de dados	3
2.3	Convertendo tipos	4
2.3.1	Convertendo para objetos string - <code>.astype()</code>	5
2.3.2	Convertendo para valores numéricos	6
2.3.2.1	<code>to_numeric</code>	7
2.3.2.2	Parâmetro <code>downcast</code> de <code>to_numeric</code>	12
2.4	Dados categorizados	14
2.4.1	Conversão para categoria	14
2.4.2	Manipulando dados categorizados	15
2.4.2.1	Atributos de Consulta	16
2.4.2.2	Modificação de Rótulos e Estrutura	18
2.4.2.3	Gestão de Ordem e Reconfiguração	22
2.5	Conclusão	25
3	Strings e dados do tipo texto	26
3.1	Introdução	26
	Objetivo	26
3.2	<code>strings</code>	27
3.2.1	Obtendo subconjuntos e fatiando strings	27
3.2.1.1	Uma letra	28
3.2.1.2	Fatiando várias letras	28
3.2.1.3	Números negativos	29
3.2.2	Obtendo o último caractere de uma string	30
3.2.2.1	Fatiando a partir do início ou até o fim	31
3.2.2.2	Fatiando com incrementos	32
3.3	Métodos de <code>string</code>	33
3.4	Outros métodos de <code>string</code>	35
3.4.1	Método <code>join</code>	35
3.4.2	Método <code>splitlines</code>	36
3.5	Formatação de <code>strings</code>	39
3.5.1	Formatação de <code>strings</code> personalizada	39
	Formatação de Strings Personalizada (<code>string.Formatter</code>)	39
	<code>parse(format_string)</code>	40
	<code>get_field()</code>	41
	<code>get_value(key, args, kwargs)</code>	42

	<code>format_field(value, format_spec)</code>	42
	<code>convert_field(value, conversion)</code>	42
3.5.2	Formatação de <code>strings</code> de caracteres	43
3.5.3	Formatação de números	46
3.5.4	Formatação no estilo do <code>printf</code> de C	48
3.5.5	Strings literais formatadas em Python 3.6+ - f-strings	49
3.6	Expressões regulares (RegEx)	50
3.6.1	Correspondência de padrão	51
3.6.2	Encontrando um padrão	51
3.6.3	Substituindo um padrão	51
3.6.4	Compilando um padrão	51
3.7	Biblioteca regex	52
3.8	Conclusão	52
4	Apply	53
5	Operações groupby: separar-aplicar-combinar	53

LISTA DE FIGURAS

LISTA DE TABELAS

1	Tabela de Referência Rápida de Tipos de Dados no Pandas	2
2	Diferenças entre <code>astype</code> e <code>to_numeric</code> no Pandas	4
3	Comportamento do parâmetro <code>errors</code> no <code>pd.to_numeric()</code>	9
4	Comportamento do parâmetro <code>downcast</code> no <code>pd.to_numeric()</code>	13
5	API de categoria.	15
6	Posições dos índices para a string ‘grail’	27
7	Posições dos índices para a string “a scratch”	27
8	Métodos de string de Python	33
9	Exemplos de uso dos métodos de string de Python	34
10	Métodos para Customização (Subclasses)	40

1 Objetivo

O objetivo deste estudo é explorar e documentar as funcionalidades essenciais das principais bibliotecas científicas do Python, como **NumPy**, **Pandas** e outras, através de exemplos práticos e casos de uso selecionados. Pretende-se consolidar o conhecimento sobre a manipulação, análise e visualização de dados, servindo como um guia de referência pessoal para futuros projetos de programação científica.

2 Tipos de dados

2.1 Introdução

Os tipos de dados determinam o que pode e o que não pode ser feito com uma variável (isto é, coluna).

Por exemplo, quando tipos de dados numéricos são somados, o resultado será uma soma dos valores; por outro lado, se trings (no Pandas, elas são chamadas de `object`) forem somadas, elas serão concatenadas.

Este capítulo apresenta uma visão geral rápida dos diversos tipos de dados que você poderá encontrar no Pandas e as formas de converter um tipo de dado em outro.

Tabela 1: Tabela de Referência Rápida de Tipos de Dados no Pandas

Tipo no Pandas	O que representa	Exemplos de Uso
<code>int64 / int32</code>	Números inteiros (sem casas decimais).	Idades, quantidades, anos, IDs.
<code>float64 / float32</code>	Números decimais (ponto flutuante).	Preços, medidas, taxas, coordenadas.
<code>object</code>	Geralmente strings ou tipos mistos.	Nomes, endereços, descrições.
<code>bool</code>	Valores lógicos (Verdadeiro ou Falso).	Flags (ex: 'ativo', 'pago', 'concluido').
<code>datetime64</code>	Representação de datas e horários.	Data de venda, timestamp de log, nascimento.
<code>category</code>	Variáveis categóricas (lista finita de valores).	Gênero, UF, níveis de escolaridade.

Objetivos

Este capítulo abordará:

1. Como encontrar os tipos de dados das colunas em um dataframe.
2. Conversão entre diversos tipos de dados.
3. Como trabalhar com tipos de dados categorizados.

Em outro capítulo discutirá como trabalhar com datas e horas.

2.2 Tipos de dados

Neste capítulo, usaremos o conjunto de dados embutidos `tips` do `seaborn`.

```
import seaborn as sns
import pandas as pd

tips = sns.load_dataset("tips")
```

Para obter uma lista dos tipos de dados armazenados em cada coluna de nosso dataframe, chamamos o atributo `dtypes`.

```
print(tips.dtypes)
```

```
total_bill    float64
tip           float64
sex           category
smoker        category
day           category
time          category
size          int64
dtype: object
```

Anteriormente já foi listado os diversos tipos de dados que podem ser armazenados em uma coluna do Pandas. Nosso conjunto de dados inclui os tipos de dados `int64`, `float64` e `category`.

Os tipos `int64` e `float64` representam valores numéricos sem e com casas decimais, respectivamente. O número após o tipo de dado numérico representa a quantidade de bits de informação que será armazenada para aquele número em particular.

O tipo de dado `category` representa variáveis categorizadas e difere do tipo de dado `object` genérico, que armazena `strings` arbitrárias. Exploraremos essas diferenças mais adiante neste capítulo.

Como o conjunto de dados `tips` é um conjunto totalmente preparado e limpo, as variáveis que armazenam `strings` foram salvas com uma categoria (`category`).

2.3 Convertendo tipos

O tipo de dado armazenado em uma coluna determinará os tipos de funções e cálculos que podemos realizar nos dados que se encontram nessa coluna. Então sem dúvida é importante saber como fazer conversões entre tipos de dados.

Esta seção tem como foco o modo de converter de um tipo de dado para outro; tenha em mente, porém, que não é mandatório fazer todas as conversões de tipos de dados de uma só vez, no momento em que você recebe seus dados. A análise de dados não é um processo linear e é possível optar por fazer a conversão de tipos durante a análise, conforme necessário. Já vimos um exemplo disto, quando tentamos converter um valor de data, transformando-o somente nos números dos anos.

Tabela 2: Diferenças entre `astype` e `to_numeric` no Pandas

Característica	<code>.astype()</code>	<code>pd.to_numeric()</code>
Versatilidade	Converte para qualquer tipo (int, float, string, category, etc).	Focada exclusivamente em números (int ou float).
Tratamento de Erros	Rígido. Se encontrar uma letra em uma coluna de números, o código trava.	Flexível. Possui o parâmetro <code>errors='coerce'</code> para lidar com sujeira.
Valores Inválidos	Frequentemente resulta em <code>ValueError</code> se houver espaços ou símbolos.	Transforma sujeira (ex: 'vazio', '??') em NaN automaticamente.
Sintaxe	Método do Series/DataFrame (<code>df['col'].astype()</code>)	Função do pandas (<code>pd.to_numeric(df['col'])</code>)
Uso Recomendado	Quando os dados estão limpos ou para tipos <code>str</code> e <code>category</code> .	Quando os dados vêm de fontes externas e podem conter erros.

2.3.1 Convertendo para objetos string - `.astype()`

Em nossos dados de `tips`, as variáveis `sex`, `smoker`, `day` e `time` estão armazenadas como `category`.

Em geral, é muito mais fácil trabalhar com tipos `object` para string quando a variável não é um número.

Alguns conjuntos de dados podem ter uma coluna `id` na qual esse dado é armazenado como um número, mas que não terá nenhum significado se você fizer um cálculo nesse valor (por exemplo, se tentar achar a média). Identificadores únicos ou números de `id` em geral são codificados dessa forma, e você talvez queira convertê-los em tipos `object` para string de acordo com a sua necessidade.

Para converter valores em strings, usamos o método `astype` na coluna. `astype` aceita um parâmetro `dtype` que será o novo tipo de dado assumido pela coluna. Nesse caso, queremos converter a variável `sex` em um `object` para string `str`.

```
tips['sex_str'] = tips['sex'].astype(str)
```

Python tem tipos embutidos `str`, `float`, `int`, `complex` e `bool`. No entanto, é possível também especificar qualquer `dtype` da biblioteca `numpy`.

Se observarmos os `dtypes` agora, veremos que `Unique Key` tem um `dtype` igual ao `object`.

```
print(tips.dtypes)
```

```
total_bill    float64
tip           float64
sex           category
smoker        category
day           category
time          category
size          int64
sex_str       str
dtype: object
```

2.3.2 Convertendo para valores numéricos

O método `astype` é uma função genérica que pode ser usada para converter qualquer coluna de um dataframe em outro `dtype`.

Lembre-se de que cada coluna é um objeto `Series` do Pandas - é por isso que a documentação do `astype` está em `pandas.Series.astype`. O exemplo, nesse caso, mostra como alterar o tipo de uma coluna do dataframe, mas, se você estiver trabalhando com um objeto `Series`, poderá usar o mesmo método `astype` para convertê-lo também.

É possível especificar qualquer tipo embutido ou do `numpy` para o método `astype` converter o `dtype` da coluna. Por exemplo, se quiséssemos converter a coluna `total_bill` em um `object` para string primeiro, e então de volta para o seu `float64` original, poderíamos passar `str` e `float` para o `astype`, respectivamente.

```
# Converte total_bill em uma string
tips['total_bill'] = tips['total_bill'].astype(str)
print(tips.dtypes)
```

```
total_bill    str
tip           float64
sex           category
smoker        category
day           category
time          category
size          int64
sex_str       str
dtype: object
```

```
# Converte de volta para um float
tips['total_bill'] = tips['total_bill'].astype(float)
print(tips.dtypes)
```

```
total_bill    float64
tip           float64
sex           category
smoker        category
day           category
time          category
size          int64
sex_str       str
dtype: object
```

2.3.2.1 to_numeric

Ao converter variáveis em valores numéricos (por exemplo, `int` e `float`), a função `to_numeric` do Pandas, a qual trata melhor os valores não numéricos, também pode ser usada.

Como uma coluna em um dataframe deve ter o mesmo `dtype`, haverá ocasiões em que uma coluna numérica conterá strings como alguns de seus valores. Por exemplo, em vez do valor `NaN` que representa um valor ausente no Pandas, uma coluna numérica poderia usar a string `'missing'` ou `'null'` para essa finalidade. Isso faria com que toda a coluna fosse uma string do tipo `object` em vez de ser um tipo numérico.

Vamos obter um subconjunto de nosso dataframe `tips` e colocar também um valor `'missing'` na coluna `total_bill` para mostrar como a função `to_numeric` atua.

```
# Obtém um subconjunto de tips
tips_sub_miss = tips.head(10)

# 1. Converte a coluna para o tipo 'object' (string)
tips_sub_miss['total_bill'] = tips_sub_miss['total_bill'].astype(object)

# Atribui alguns valores ausentes ('missing')
tips_sub_miss.loc[[1,3,5,7], 'total_bill'] = 'missing'

print(tips_sub_miss)
```

	total_bill	tip	sex	smoker	day	time	size	sex_str
0	16.99	1.01	Female	No	Sun	Dinner	2	Female
1	missing	1.66	Male	No	Sun	Dinner	3	Male
2	21.01	3.50	Male	No	Sun	Dinner	3	Male
3	missing	3.31	Male	No	Sun	Dinner	2	Male
4	24.59	3.61	Female	No	Sun	Dinner	4	Female
5	missing	4.71	Male	No	Sun	Dinner	4	Male
6	8.77	2.00	Male	No	Sun	Dinner	2	Male
7	missing	3.12	Male	No	Sun	Dinner	4	Male
8	15.04	1.96	Male	No	Sun	Dinner	2	Male
9	14.78	3.23	Male	No	Sun	Dinner	2	Male

Ao observar os `dtypes`, você verá que a coluna `total_bill` agora é uma string `object`.

```
print(tips_sub_miss.dtypes)
```

```
total_bill    object
tip           float64
sex           category
smoker        category
day           category
time          category
size          int64
sex_str       str
dtype: object
```

Erro

Se tentarmos agora usar o método `astype` para converter a coluna de volta em um `float`, veremos um erro: O Pandas não sabe como converter ‘missing’ em um `float`.

```
# Isto causará um erro
tips_sub_miss['total_bill'].astype(float)
```

Se usarmos a função `to_numeric` da biblioteca Pandas, veremos um erro semelhante.

```
# Isto causará um erro
pd.to_numeric(tips_sub_miss['total_bill'])
```

Parâmetro `errors`

`to_numeric` tem um parâmetro chamado `errors` que determina o que acontece quando a função encontra um valor que ela consiga converter para um valor numérico. Por padrão, é definido com ‘raise’, ou seja, se a função encontrar um valor que não possa ser convertido em um valor numérico, um erro será gerado.

De acordo com a documentação, há três valores possíveis para `errors`:

1. ‘raise’ (default) gerará um erro se a conversão para um valor numérico não puder ser feita.
2. ‘coerce’ devolverá NaN para valores que não puderem ser convertidos para valores numéricos.
3. ‘ignore’ devolverá o vetor sem converter a coluna em um valor numérico (isto é, não fará nada).

Tabela 3: Comportamento do parâmetro `errors` no `pd.to_numeric()`

Opção do Parâmetro	Comportamento Técnica	Melhor Caso de Uso
'raise' (Padrão)	Interrompe a execução e exibe um erro (ValueError).	Dados que você sabe que devem ser 100% numéricos.
'coerce'	Transforma valores não numéricos em NaN (Not a Number).	Dados sujos (ex: 'S/N', '-', 'falta') que precisam ser limpos.
'ignore'	Mantém o valor original e a coluna permanece como object .	Raro. Útil se você quer tentar converter, mas manter tudo intacto se falhar.

ignore

Saindo da ordem da documentação, se passarmos o valor 'ignore' para **errors**, nada mudará em nossa coluna. No entanto, não veremos tampouco uma mensagem de erro - o que nem sempre poderá ser o comportamento desejado.

```
#| warning: false
tips_sub_miss['total_bill'] = pd.to_numeric(
    tips_sub_miss['total_bill'],
    errors='ignore'
)
```

```
print(tips_sub_miss)
```

```
print(tips_sub_miss.dtypes)
```

Obs.: ignore foi descontinuado na atualização mais recente do Pandas.

coerce

Em comparação, se passarmos o valor 'coerce', teremos valores NaN para a sting 'missing'.

```
tips_sub_miss['total_bill'] = pd.to_numeric(
    tips_sub_miss['total_bill'],
    errors='coerce'
)

print(tips_sub_miss)
```

	total_bill	tip	sex	smoker	day	time	size	sex_str
0	16.99	1.01	Female	No	Sun	Dinner	2	Female
1	NaN	1.66	Male	No	Sun	Dinner	3	Male
2	21.01	3.50	Male	No	Sun	Dinner	3	Male
3	NaN	3.31	Male	No	Sun	Dinner	2	Male
4	24.59	3.61	Female	No	Sun	Dinner	4	Female
5	NaN	4.71	Male	No	Sun	Dinner	4	Male
6	8.77	2.00	Male	No	Sun	Dinner	2	Male
7	NaN	3.12	Male	No	Sun	Dinner	4	Male
8	15.04	1.96	Male	No	Sun	Dinner	2	Male
9	14.78	3.23	Male	No	Sun	Dinner	2	Male

```
print(tips_sub_miss.dtypes)
```

```
total_bill    float64
tip           float64
sex           category
smoker        category
day           category
time          category
size          int64
sex_str       str
dtype: object
```

Esse é um truque útil quando você sabe que uma coluna deve conter valores numéricos, mas, por algum motivo, os dados incluem valores não numéricos.

2.3.2.2 Parâmetro downcast de to_numeric

A função `to_numeric` tem outro parâmetro chamado `downcast`, que permite alterar o `dtype` numérico para o menor `dtype` numérico possível (isto é, fazer um “downcast”) depois que uma coluna (ou vetor) tiver sido convertida com sucesso em um vetor numérico.

Por padrão, o valor é definido com `None`, mas outros valores possíveis são `‘integer’`, `‘signed’`, `‘unsigned’` e `‘float’`.

Compare os `dtypes` depois de termos especificado o argumento `downcast`.

```
tips_sub_miss['total_bill'] = pd.to_numeric(
    tips_sub_miss['total_bill'],
    errors='coerce',
    downcast='float'
)

print(tips_sub_miss)
```

	total_bill	tip	sex	smoker	day	time	size	sex_str
0	16.99	1.01	Female	No	Sun	Dinner	2	Female
1	NaN	1.66	Male	No	Sun	Dinner	3	Male
2	21.01	3.50	Male	No	Sun	Dinner	3	Male
3	NaN	3.31	Male	No	Sun	Dinner	2	Male
4	24.59	3.61	Female	No	Sun	Dinner	4	Female
5	NaN	4.71	Male	No	Sun	Dinner	4	Male
6	8.77	2.00	Male	No	Sun	Dinner	2	Male
7	NaN	3.12	Male	No	Sun	Dinner	4	Male
8	15.04	1.96	Male	No	Sun	Dinner	2	Male
9	14.78	3.23	Male	No	Sun	Dinner	2	Male

```
print(tips_sub_miss.dtypes)
```

```
total_bill    float32
tip           float64
sex           category
smoker        category
day           category
time          category
size          int64
sex_str       str
dtype: object
```

Podemos ver que o `dtypes` de nossa coluna não é mais `'float64'`, isto é, agora ela ocupa um espaço menor (de memória), pois sofreu um `downcast` para `float32`.

Tabela 4: Comportamento do parâmetro `downcast` no `pd.to_numeric()`

Opção do Parâmetro	Comportamento Técnico	Melhor Caso de Uso
<code>'integer'</code>	Reduz para o menor tipo inteiro possível (ex: <code>int8</code> , <code>int16</code>).	Economia agressiva de memória em colunas de contagem.
<code>'signed'</code>	Foca no menor tipo de inteiro que suporte números negativos.	Dados numéricos inteiros que possuem variação de sinal.
<code>'unsigned'</code>	Reduz para o menor inteiro positivo (sem sinal).	IDs, idades ou qualquer métrica que nunca seja negativa.
<code>'float'</code>	Reduz a precisão de 64 bits para 32 bits (<code>float32</code>).	Datasets gigantes de ponto flutuante para reduzir o uso de RAM.

2.4 Dados categorizados

Nem todos os valores são numéricos. O Pandas tem um `dtype category` capaz de codificar valores de categoria.

Eis alguns casos de uso para dados categorizados:

1. Armazenar dados desse modo é mais eficaz quanto a memória e á velocidade, particularmente se o conjunto de dados incluir muitos valores repetidos de string.
2. Dados categorizados podem ser apropriados quando uma coluna de valores tiver uma ordem (por exemplo, uma escala de Linkert).
3. Algumas bibliotecas Python sabem lidar com dados categorizados (por exemplo, na adequação de modelos estatísticos).

2.4.1 Conversão para categoria

Para converter uma coluna em um tipo categoria, passamos `category` ao método `astype`.

```
# Converte a coluna sex em uma string object antes
tips['sex'] = tips['sex'].astype('str')
print(tips.info())
```

```
<class 'pandas.DataFrame'>
RangeIndex: 244 entries, 0 to 243
Data columns (total 8 columns):
#   Column      Non-Null Count  Dtype
---  -
0   total_bill  244 non-null    float64
1   tip         244 non-null    float64
2   sex         244 non-null    str
3   smoker      244 non-null    category
4   day         244 non-null    category
5   time        244 non-null    category
6   size        244 non-null    int64
7   sex_str     244 non-null    str
dtypes: category(3), float64(2), int64(1), str(2)
memory usage: 10.8 KB
None
```

```
# Converte a coluna sex de volta para dados categorizados
tips['sex'] = tips['sex'].astype('category')
print(tips.info())
```

```
<class 'pandas.DataFrame'>
RangeIndex: 244 entries, 0 to 243
Data columns (total 8 columns):
#   Column      Non-Null Count  Dtype
---  -
0   total_bill  244 non-null    float64
1   tip         244 non-null    float64
2   sex         244 non-null    category
3   smoker      244 non-null    category
4   day          244 non-null    category
5   time         244 non-null    category
6   size         244 non-null    int64
7   sex_str      244 non-null    str
dtypes: category(4), float64(2), int64(1), str(1)
memory usage: 9.3 KB
None
```

2.4.2 Manipulando dados categorizados

A API de referência tem uma lista das operações que podem ser realizadas em uma **Series** categorizada. A lista foi reproduzida na Tabela 5.

Tabela 5: API de categoria.

Atributo ou método	Descrição
<code>Series.cat.categories</code>	As categorias.
<code>Series.cat.ordered</code>	Informa se as categorias estão ordenadas.
<code>Series.cat.codes</code>	Devolve o código inteiro da categoria.
<code>Series.cat.rename_categories()</code>	Renomeia as categorias.
<code>Series.cat.reorder_categories()</code>	Reordena as categorias.
<code>Series.cat.add_categories()</code>	Adiciona novas categorias.
<code>Series.cat.remove_categories()</code>	Remove categorias.
<code>Series.cat.remove_unused_categories()</code>	Remove categorias não usadas.
<code>Series.cat.set_categories()</code>	Define novas categorias.
<code>Series.cat.as_ordered()</code>	Deixa as categorias ordenadas.
<code>Series.cat.as_unordered()</code>	Deixa as categorias não ordenadas.

2.4.2.1 Atributos de Consulta

- `Series.cat.categories`:

Retorna o índice com os nomes das categorias existentes. É útil para saber exatamente quais rótulos o Pandas está reconhecendo (ex: “Jovem”, “Adulto”, “Idoso”).

```
import pandas as pd

# Criando o DataFrame
df = pd.DataFrame({'tamanho': ['M', 'P', 'G', 'P', 'M']})

# Convertendo para o tipo categórico
df['tamanho'] = df['tamanho'].astype('category')

print(df)
```

```
   tamanho
0        M
1        P
2        G
3        P
4        M
```

```
print(df.dtypes)
```

```
tamanho    category
dtype: object
```

```
print(df['tamanho'].cat.categories)
```

```
Index(['G', 'M', 'P'], dtype='str')
```

- `Series.cat.ordered`:

Retorna um valor booleano (True ou False). Diz se a coluna entende que existe uma hierarquia (ex: se “P” vem antes de “M”).

```
print(df['tamanho'].cat.ordered)
# Saída: False
```

```
False
```

- `Series.cat.codes`:

Transforma os nomes em números. Se você tem “Sim” e “Não”, ele transforma em 0 e 1. É o que os modelos de Machine Learning costumam usar por baixo dos panos.

Este é o mais interessante. Ele mostra a representação numérica (o “RG” de cada categoria).

```
print(df['tamanho'].cat.codes)
```

```
0    1
1    2
2    0
3    2
4    1
dtype: int8
```

2.4.2.2 Modificação de Rótulos e Estrutura

- `Series.cat.rename_categories()`:

Altera apenas o nome dos rótulos. Se você tem categorias em inglês e quer passar para português, usa esta função.

Serve para trocar o nome dos rótulos existentes. Você pode passar uma lista (na ordem que aparecem em `.categories`) ou um dicionário.

```
# Trocando P, M, G por Pequeno, Médio, Grande
df['tamanho'] = df['tamanho'].cat.rename_categories({'P': 'Pequeno',\
    'M': 'Médio', 'G': 'Grande'})

print(df['tamanho'].cat.categories)
# Saída: Index(['Grande', 'Médio', 'Pequeno'], dtype='object')
```

```
Index(['Grande', 'Médio', 'Pequeno'], dtype='str')
```

- `Series.cat.add_categories()`:

“Abre espaço” para novos nomes. Se a coluna só aceita “A” e “B”, e você tentar inserir “C” sem usar isso antes, o Pandas vai gerar um erro ou transformar o “C” em nulo (NaN).

Se você tentar mudar um valor para ‘GG’ em uma coluna que só conhece ‘P’, ‘M’ e ‘G’, o Pandas vai dar erro ou colocar um nulo. Você precisa “autorizar” o novo valor primeiro.

```
# Adicionando o tamanho 'GG' à lista de permitidos
df['tamanho'] = df['tamanho'].cat.add_categories(['GG'])

# Agora eu posso, por exemplo, mudar a primeira linha para GG
df.loc[0, 'tamanho'] = 'GG'
print(df['tamanho'].unique())
# Saída: ['GG', 'Pequeno', 'Grande'] ...
```

```
['GG', 'Pequeno', 'Grande', 'Médio']
Categories (4, str): ['Grande', 'Médio', 'Pequeno', 'GG']
```

- `Series.cat.remove_categories()`:

Faz o oposto: proíbe um valor. Se houver dados com esse valor na tabela, eles viram NaN (vazio).

Remove categorias específicas da lista de permissões. Se houver dados marcados com a categoria removida, eles viram NaN.

```
# Removendo o tamanho 'Grande'
df['tamanho'] = df['tamanho'].cat.remove_categories(['Grande'])

# Qualquer linha que era 'Grande' agora é NaN (Not a Number)
print(df)
```

```
      tamanho
0          GG
1  Pequeno
2         NaN
3  Pequeno
4     Médio
```

- `Series.cat.remove_unused_categories()`:

É a “faxina”. Se você filtrou seu DataFrame e não existe mais ninguém da categoria “Idoso”, essa função remove esse rótulo da memória da coluna.

Exemplo:

Imagine que você tem uma loja que vende sorvete de Chocolate, Morango e Baunilha.

- Você tem uma tabela com 100 vendas.
- Você decide parar de vender Morango e apaga todas as linhas de Morango da sua tabela.
- Agora sua tabela só tem Chocolate e Baunilha.

O problema: Mesmo que não exista mais nenhum “Morango” nas linhas da tabela, o Pandas ainda guarda a “opção” Morango na estrutura da coluna. É como se o formulário ainda tivesse o quadradinho do Morango lá, mas ninguém nunca marcasse.

```
# 1. Criamos os dados
cores = pd.Series(['Azul', 'Vermelho', 'Azul'], dtype='category')
print(cores.cat.categories)
# Saída: Index(['Azul', 'Vermelho'], dtype='object')
```

```
Index(['Azul', 'Vermelho'], dtype='str')
```

```
# 2. Agora vamos remover todas as linhas que têm 'Vermelho'
cores_filtradas = cores[cores != 'Vermelho']

print(cores_filtradas)
```

```
0    Azul
2    Azul
dtype: category
Categories (2, str): ['Azul', 'Vermelho']
```

```
# A tabela só tem 'Azul', certo? Mas olha as categorias:
print(cores_filtradas.cat.categories)
# Saída: Index(['Azul', 'Vermelho'], dtype='object')
# O 'Vermelho' ainda está "assombrando" a coluna!
```

```
Index(['Azul', 'Vermelho'], dtype='str')
```

A solução: `remove_unused_categories()`

Ela olha para a tabela, vê o que realmente está escrito lá e joga fora as opções que sobraram no “checklist” mas não estão em linha nenhuma.

```
cores_filtradas = cores_filtradas.cat.remove_unused_categories()

print(cores_filtradas.cat.categories)
# Saída: Index(['Azul'], dtype='object')
# Agora sim! O 'Vermelho' sumiu de vez da estrutura.
```

```
Index(['Azul'], dtype='str')
```

2.4.2.3 Gestão de Ordem e Reconfiguração

Esta é a parte mais poderosa das categorias no Pandas. É aqui que você ensina ao computador que “Grande” é maior que “Pequeno”, algo que ele não sabe sozinho apenas lendo o texto.

Vamos usar um exemplo de Níveis de Urgência: ['Baixa', 'Média', 'Alta'].

```
df = pd.DataFrame({'urgencia': ['Alta', 'Baixa', 'Média', 'Baixa']})
df['urgencia'] = df['urgencia'].astype('category')

print(df)
```

```
urgencia
0      Alta
1     Baixa
2     Média
3     Baixa
```

```
print(df.dtypes)
```

```
urgencia    category
dtype: object
```

- `Series.cat.reorder_categories()`:

Muda a sequência das categorias. Útil para quando a ordem importa (ex: mudar de [Baixo, Alto, Médio] para [Baixo, Médio, Alto]). As categorias devem ser as mesmas que já existem.

```
# Definindo a ordem correta
df['urgencia'] = df['urgencia'].\
    cat.reorder_categories(['Baixa', 'Média', 'Alta'],\
        ordered=True)

# Agora o Pandas sabe que Baixa < Média < Alta.
# Se você ordenar o DataFrame, ele seguirá essa lógica, não a alfabética!
df = df.sort_values('urgencia')
print(df)
```

```
urgencia
1     Baixa
3     Baixa
2     Média
0      Alta
```

- `Series.cat.set_categories()`:

É a função “faz-tudo”. Com ela, você pode adicionar, remover e ordenar categorias de uma só vez, definindo exatamente como a lista final deve ser. Ela faz o trabalho de `add`, `remove` e `reorder` de uma só vez. Você passa uma lista nova e ela ajusta a coluna para ser exatamente igual a essa lista.

```
# Imagine que queremos mudar as opções para
# ['Minima', 'Baixa', 'Média', 'Alta', 'Urgente']
novas_opcoes = ['Minima', 'Baixa', 'Média', 'Alta', 'Urgente']

df['urgencia'] = df['urgencia'].cat.set_categories(novas_opcoes, ordered=True)

print(df['urgencia'].cat.categories)
```

```
Index(['Minima', 'Baixa', 'Média', 'Alta', 'Urgente'], dtype='str')
```

Atenção: Se você usar `set_categories` e esquecer de incluir uma categoria que já estava nos dados, o Pandas vai transformar esses valores em NaN.

- `Series.cat.as_ordered()`:

Força a coluna a ser ordenada. A partir daqui, o Pandas entende que faz sentido dizer que um valor é “maior” que outro.

Transforma uma categoria comum em uma categoria ordinal. Ela avisa ao Pandas: “Ei, a ordem em que os nomes aparecem na lista de categorias importa!”.

```
df['urgencia'] = df['urgencia'].cat.as_ordered()
print(df['urgencia'].cat.ordered)
```

```
True
```

```
print(df['urgencia'])
```

```
1    Baixa
3    Baixa
2    Média
0     Alta
```

```
Name: urgencia, dtype: category
```

```
Categories (5, str): ['Minima' < 'Baixa' < 'Média' < 'Alta' < 'Urgente']
```

- `Series.cat.as_unordered()`:

É o caminho reverso. Ela remove a hierarquia, voltando a tratar as categorias apenas como rótulos sem relação de “maior” ou “menor”.

```
df['urgencia'] = df['urgencia'].cat.as_unordered()

print(df['urgencia'])
```

1 Baixa

3 Baixa

2 Média

0 Alta

Name: urgencia, dtype: category

Categories (5, str): ['Minima', 'Baixa', 'Média', 'Alta', 'Urgente']

2.5 Conclusão

Este capítulo abordou a conversão de um tipo de dado em outro.

Os `dtypes` determinam quais operações podem e não podem ser executadas em uma coluna. Embora este capítulo seja relativamente curto, a conversão de tipos é uma habilidade importante quando trabalhamos com dados e usamos outros métodos do Pandas.

3 Strings e dados do tipo texto

3.1 Introdução

A maior parte dos dados presentes no mundo pode ser armazenado como texto ou strings. Até mesmo valores que poderão, em algum momento, se tornar numéricos talvez cheguem na forma de texto inicialmente. É importante ser capaz de trabalhar com dados do tipo texto.

Este capítulo não é específico do Pandas, ou seja, exploraremos principalmente como manipular strings em Python sem o Pandas. Os próximos capítulos discutirão outros conteúdos do Pandas. Então retornaremos às strings e veremos como tudo se relaciona com o Pandas.

Como informação adicional, alguns dos exemplos de string neste capítulo são provenientes de *Monty Python and the Holy Grail* (Monte Python: em busca do cálice sagrado).

Objetivo

Este capítulo abordará:

1. Obtenção de subconjuntos de `string`
2. Métodos de `string`
3. Formatação de `string`
4. Expressões regulares

3.2 strings

Em Python uma string é simplesmente uma série de caracteres. Elas são criadas por um conjunto de aspas de abertura, simples ou duplas, e as aspas de fechamento correspondentes.

Por exemplo:

```
word = 'grail'
sent = 'a scratch'
```

Criará duas strings, 'grail' e a 'scratch', e elas são atribuídas às variáveis `word` e `sent`, respectivamente.

3.2.1 Obtendo subconjuntos e fatiando strings

Podemos pensar nas strings como um contêiner de caracteres. Você pode obter um subconjunto de uma string como faria com qualquer outro contêiner Python (por exemplo, `list` ou `Series`).

A Tabela 6 e Tabela 7 mostram as strings com seus índices associados. Essas informações ajudarão você a compreender os exemplos em que fatiamos valores usando o índice.

Tabela 6: Posições dos índices para a string 'grail'

Índice Positivo	0	1	2	3	4
String	g	r	a	i	l
Índice Negativo	-5	-4	-3	-2	-1

Tabela 7: Posições dos índices para a string "a scratch"

Índice Positivo	0	1	2	3	4	5	6	7	8
String	a		s	c	r	a	t	c	h
Índice Negativo	-9	-8	-7	-6	-5	-4	-3	-2	-1

3.2.1.1 Uma letra

Para obter a primeira letra de nossas strings, podemos usar a notação de colchetes, []. Essa notação corresponde ao mesmo método que usamos quando observamos várias fatias de dados.

```
print(word[0])
```

g

```
print(sent[0])
```

a

3.2.1.2 Fatiando várias letras

Como alternativa, podemos usar a notação de fatiamento para obter intervalos de nossas strings.

```
# Obtém os 3 primeiros caracteres:  
# Note que o índice 3 é, na verdade, o 4º caractere  
  
print(word[0:3])
```

gra

Lembre-se de que, quando usamos a notação de fatiamento em Python, ela é inclusiva à esquerda, mas não à direita. Em outras palavras, o valor do índice especificado em primeiro lugar será incluído, mas não o valor do índice especificado em segundo.

Por exemplo, a notação [0:3] incluirá os caracteres de 0 a 3, mas não o índice 3. Outra maneira de dizer isso é afirmar que [0:3] incluirá os índices de 0 a 2, inclusive.

3.2.1.3 Números negativos

Lembre-se de que, em Python, passar um índice negativo faz a contagem ser iniciada a partir do **fim** de um contêiner.

```
# Obtém a última letra  
  
print(sent[-1])
```

h

O índice negativo refere-se também à posição de índice, de modo que você poderá usá-lo para fatiar valores.

```
# Obtém 'a'  
  
print(sent[-9:-8])
```

a

Números não negativos podem ser combinados com números negativos.

```
# Obtém 'a'  
  
print(sent[0:-8])
```

a

Observe que não é possível realmente obter a última letra ao usar fatiamento com índice negativo.

```
# scratch  
  
print(sent[2:-1])
```

scratc

```
# scratch  
  
print(sent[-7:-1])
```

scratc

3.2.2 Obtendo o último caractere de uma string

Obter somente o último elemento de uma string (ou qualquer outro contêiner) pode ser feito com o índice negativo -1. No entanto, isso se torna problemático quando queremos usar a notação de fatiamento e incluir também o último caractere.

Por exemplo, se tentarmos usar a notação de fatiamento para obter a palavra “scratch” da variável `sent`, o resultado devolvido teria uma letra a menos.

Como o Python não é inclusivo à direita, temos de especificar uma posição de índice igual ao último índice mais um. Para isso, podemos obter o tamanho (`len`) da string e então passar esse valor para a notação de fatiamento.

```
# Observe que o último índice é uma posição menor que
#o número devolvido len
# sent vai do 0 ao 8 caracteres
s_len = len(sent)
print(s_len)
```

9

```
print(sent[2:s_len])
```

scratch

3.2.2.1 Fatiando a partir do início ou até o fim

Uma tarefa muito comum é fatiar um valor a partir do início até certo ponto de uma string (ou contêiner). O primeiro elemento é sempre 0, portanto será sempre possível escrever algo como `word[0:3]` para obter os três primeiros elementos, ou `word[-3:length(word)]` para obter os três últimos.

Outro atalho para essa tarefa é deixar em branco os dados à esquerda ou à direita de:

- Se o lado esquerdo de `:` estiver vazio, a fatia será tomada do início e terminará no índice à direita (não inclusivo).
- Se o lado direito de `:` estiver vazio, a fatia terá início à esquerda e terminará no final da string.

Por exemplo:

```
print(word[0:3])
```

gra

```
print(word[:3])
```

gra

Por exemplo, as fatias a seguir equivalentes:

```
print(sent[2:len(sent)])
```

scratch

```
print(sent[2:])
```

scratch

Outra forma de especificar a string toda é deixar os dois valores vazios.

```
print(sent[:])
```

a scratch

3.2.2.2 Fatiando com incrementos

A última notação para fatiamento nos permite fatiar em incrementos. Para isso, use um segundo dois-pontos, :, para especificar um terceiro número. Esse permite determinar o incremento com base no qual os valores serão extraídos.

Por exemplo, podemos obter uma string com os caracteres alternados passando 2 para considerar todo segundo caractere.

```
print(sent[::2])
```

asrth

Qualquer inteiro pode ser passado nessa posição; portanto, se você quiser considerar cada terceiro caractere (ou valor em um contêiner), passe o valor 3.

```
print(sent[::3])
```

act

3.3 Métodos de string

Muitos métodos também são usados quando processamos dados em Python. Uma lista com todos os métodos de string pode ser encontrada na página “*String Methods*” (Métodos de string) da documentação. A Tabela 8 e Tabela 9 resumem alguns métodos de string comumente utilizados em Python.

Tabela 8: Métodos de string de Python

Método	Descrição
capitalize	Converte o primeiro caractere em letra maiúscula.
count	Conta o número de ocorrências de uma string.
startswith	True se a string começar com o valor especificado.
endswith	True se a string terminar com o valor especificado.
find	Menor índice no qual há correspondência com a string; -1 se não houver correspondência.
index	É igual a find , mas devolve ValueError se não houver correspondência.
isalpha	True se todos os caracteres forem alfabéticos.
isdecimal	True se todos os caracteres forem numéricos decimais. (Consulte a documentação, assim como isdigit , isnumeric e isalnum)
isalnum	True se todos os caracteres forem alfanúmericos (alfabéticos ou numéricos).
lower	Cópia de uma string com todos as letras minúsculas.
upper	Cópia de uma string com todos as letras maiúsculas.
replace	Cópia de uma string com os valores old substituídos por new .
strip	Remove todos os espaços em branco no início e no fim: veja também lstrip e rstrip .
split	Devolve uma lista de valores separados pelo delimitador (separador).
partition	Semelhante ao split(maxsplit=1) , mas devolve também o separador.
center	Centraliza a string com uma dada largura.
zfill	Cópia da string preenchida com '0's à esquerda.

Tabela 9: Exemplos de uso dos métodos de string de Python

Código	Resultado
'black Knight'.capitalize()	'Black Knight'
'It's just a flesh wound!'.count('u')	2
'Halt! Who goes there?'.startswith('Halt')	True
'coconut'.endswith('nut')	True
'It's just a flesh wound!'.find('u')	7
'It's just a flesh wound!'.index('scratch')	ValueError
'old woman'.isalpha()	False (Há um espaço em branco)
'37'.isdecimal()	True
'I'm 37'.isalnum()	False (Apóstrofo e espaço)
'Black Knight'.lower()	'black knight'
'Black Knight'.upper()	'BLACK KNIGHT'
'flesh wound!'.replace('flesh wound', 'scratch')	scratch!
' I'm not dead. '.strip()	I'm not dead.
'NI! NI! NI!'.split(sep=' ')	['NI!', 'NI!', 'NI!', 'NI!']
'3,4'.partition(',')	('3', ',', '4')
'nine'.center(width=10)	' nine '
'9'.zfill(with=5)	00009

3.4 Outros métodos de string

Para além dos métodos da Tabela 8, este tópico lista mais alguns métodos de string que são convenientes (mas difíceis de apresentar em uma tabela).

3.4.1 Método `join`

O método `join` aceita um contêiner (por exemplo, uma `list`) e devolve uma nova string contendo cada elemento da lista. Por exemplo, suponha que queremos combinar coordenadas na notação DMS (Degrees, Minutes, Seconds, ou Graus, Minutos, Segundos).

```
d1 = '40'
m1 = '46'
s1 = '52.837'
u1 = 'N'

d2 = '73'
m2 = '58'
s2 = '26.302'
u2 = 'W'
```

Podemos juntar todos os valores com um espaço, ' ', usando o método `join` no espaço.

```
coords = ' '.join([d1, m1, s1, u1,\
    d2, m2, s2, u2])

print(coords)
```

```
40 46 52.837 N 73 58 26.302 W
```

Esse método também será útil se houver uma lista de strings que você queira separar usando o seu próprio delimitador (por exemplo, tabulação ; vírgulas ,).

Se quiséssemos, poderíamos usar `split` em um espaço, ' ', e obter as partes individuais de `coords`.

3.4.2 Método `splitlines`

O método `splitlines` é semelhante ao método `split`. Em geral, é usado em strings com várias linhas e devolve uma lista em que cada elemento será uma linha da string multilinha.

```
multi_str = """Guard: What? Ridden on a horse?
King Arthur: Yes!
Guard: You're using coconuts!
King Arthur: What?
Guard: You've got ... coconut[s] and you're bangin' 'em together.
"""

print(multi_str)
```

```
Guard: What? Ridden on a horse?
King Arthur: Yes!
Guard: You're using coconuts!
King Arthur: What?
Guard: You've got ... coconut[s] and you're bangin' 'em together.
```

Podemos obter cada linha como um elemento separado em uma lista usando `splitlines`.

```
multi_str_split = multi_str.splitlines()
pprint(multi_str_split)
```

```
['Guard: What? Ridden on a horse?',
 'King Arthur: Yes!',
 "Guard: You're using coconuts!",
 'King Arthur: What?',
 "Guard: You've got ... coconut[s] and you're bangin' 'em together."]
```

Observações:

- Biblioteca `pprint`:
 - É o módulo completo que contém ferramentas para formatação de dados. Ela analisa objetos Python (listas, dicionários, tuplas) e calcula como eles devem ser exibidos para não estourarem a largura da tela.
- função `pprint`:
 - Diferente do `print()` comum, que joga tudo em uma linha só, a função `pprint` quebra as linhas automaticamente, recua (indenta) os itens e organiza a saída visualmente.
 - Ela evita que sua lista “fuja” da margem da folha, pois ela detecta o fim da linha e empilha os dados verticalmente.

Exemplo:

Por fim, suponha que quisessémos somente o texto do **Guard**. Essa é uma conversa entre duas pessoas, portanto **Guard** fala em linhas alternadas.

```
guard = multi_str_split[:,2]
pprint(guard)
```

```
['Guard: What? Ridden on a horse?',
 'Guard: You're using coconuts!',
 'Guard: You've got ... coconut[s] and you're bangin' 'em together."]
```

Há algumas maneiras para obter somente as linhas de **Guard**. Uma fonte seria usar o método **replace** na string e substituir a string **'Guard: '** por uma string vazia, isto é, **''**. Poderíamos então usar o método **splitlines**.

```
guard = multi_str.replace('Guard: ', '')\
    .splitlines()[:,2]
pprint(guard)
```

```
['What? Ridden on a horse?',
 'You're using coconuts!',
 'You've got ... coconut[s] and you're bangin' 'em together."]
```

3.5 Formatação de strings

A formatação de strings nos permite especificar um template genérico para uma string e inserir variáveis no padrão. Esse template também é capaz de lidar com várias maneiras de representar visualmente uma string - por exemplo, mostrando dois valores decimais em um float ou exibindo um número como uma porcentagem, em vez de ser um valor decimal.

A formatação de string pode ajudar até mesmo quando queremos exibir algo no console. Em vez de simplesmente exibir a variável, podemos mostrar uma string que ofereça pistas sobre o valor apresentado.

Há vários casos de uso para formatação de strings, e Python tem duas formas de tratá-las.

3.5.1 Formatação de strings personalizada

Uma forma mais novade formatação de strings está descrita na documentação¹ sobre o assunto, junto com vários exemplos².

Formatação de Strings Personalizada (`string.Formatter`)

O Python permite realizar substituições complexas de variáveis através do método `format()`. Para customizar esse comportamento, existe a classe `string.Formatter`, que utiliza a mesma lógica interna do método padrão.

Principais Métodos de Execução:

- `format()`: A interface principal. Recebe uma string de formato e argumentos (`*args`, `kwargs`).
- `vformat()`: O “motor” real da formatação. É útil quando os argumentos já estão em um dicionário predefinido, evitando desempacotar e reempacotar os dados.

¹<https://docs.python.org/3.6/library/string.html#string-formatting>

²<https://docs.python.org/3.6/library/string.html#format-examples>

Tabela 10: Métodos para Customização (Subclasses)

Método	Descrição
<code>parse()</code>	Decompõe a string em texto literal e campos de substituição.
<code>get_field()</code>	Converte o nome do campo (ex: <code>0[nome]</code>) no objeto a ser formatado.
<code>get_value()</code>	Recupera o valor do dado via índice (inteiro) ou chave (string).
<code>format_field()</code>	Executa a formatação final chamando a função nativa <code>format()</code> .
<code>convert_field()</code>	Converte o valor para tipos específicos: <code>s</code> (str), <code>r</code> (repr) ou <code>a</code> (ascii).
<code>check_unused_args()</code>	Verifica e valida se há argumentos que não foram utilizados na string.

Resumo do fluxo de execução:

O processo segue uma ordem lógica de processamento que garante a flexibilidade da formatação:

1. **Análise:** O `parse()` quebra a estrutura da string.
2. **Busca:** O `get_field()` e `get_value()` localizam os dados.
3. **Conversão:** O `convert_field()` ajusta o tipo se solicitado.
4. **Formatação:** O `format_field()` gera o resultado visual final.

`parse(format_string)`

Este método “quebra” a frase para entender o que é texto fixo e o que é variável.

```
from string import Formatter
fmt = Formatter()

texto = "Oi, {nome}! Você tem {idade} anos."
for literal, campo, espec, conv in fmt.parse(texto):
    print(f"Texto: '{literal}' | Campo: '{campo}'")
```

```
Texto: 'Oi, ' | Campo: 'nome'
Texto: '! Você tem ' | Campo: 'idade'
Texto: ' anos.' | Campo: 'None'
```

`get_field()`

Ele recebe o nome que você escreveu entre as chaves (ex: `pessoa.nome`) e o divide em duas partes:

1. **A Base:** Ele chama o `get_value` para buscar o objeto principal (a variável `pessoa`).
2. **O Resto:** Ele identifica as “instruções extras” (como `.nome` ou `[0]`) que devem ser aplicadas a esse objeto.

```
from string import Formatter

# 1. Definimos uma estrutura simples
class Carro:
    def __init__(self, modelo):
        self.modelo = modelo

meu_carro = Carro("Fusca")
fmt = Formatter()

# 2. Vamos usar o get_field manualmente para ver o que ele faz
# A string de formato que queremos testar é "{0.modelo}"
# Onde '0' é o índice do argumento e '.modelo' é o atributo.

campo_completo = "0.modelo"
args = (meu_carro,) # Passamos o objeto dentro de uma tupla
kwargs = {}

# O get_field vai decompor "0.modelo"
objeto_final, chave_usada = fmt.get_field(campo_completo, args, kwargs)

print(f"Valor extraído: {objeto_final}")
print(f"Chave principal usada: {chave_usada}")

# Saída:
# Valor extraído: Fusca
# Chave principal usada: 0
```

Valor extraído: Fusca
Chave principal usada: 0

`get_value(key, args, kwargs)`

É o responsável por “ir buscar” o valor dentro das suas variáveis.

```
# Se você passar (nome="Sergio"), o get_value busca a chave "nome"
# Se você passar ("Sergio"), o get_value busca o índice 0
kwargs = {'nome': 'Sergio'}
print(fmt.get_value('nome', (), kwargs)) # Saída: Sergio
```

Sergio

`format_field(value, format_spec)`

Aqui é onde acontece a mágica dos números e alinhamentos (como o `:.2f`).

```
numero = 10.5678
print(fmt.format_field(numero, ".2f")) # Saída: 10.57
```

10.57

`convert_field(value, conversion)`

Ele aplica transformações rápidas como `!r` (representação com aspas) ou `!s` (string pura).

```
valor = "Brasil"
# 's' para string, 'r' para repr (mostra as aspas)
print(fmt.convert_field(valor, 'r')) # Saída: 'Brasil' (com aspas)
```

'Brasil'

3.5.2 Formatação de strings de caracteres

Para formatar strings de caracteres, você deve essencialmente escrever string com caracteres placeholders especiais³ e usar o método `format` na string para inserir valores nesses placeholders.

Exemplos:

```
var = 'flash wound'
s = "It's just a {}!"

print(s.format(var))
```

It's just a flash wound!

```
print(s.format('scratch'))
```

It's just a scratch!

```
# 1. O modelo com os placeholders {}
modelo_frase = "Olá, {}! Seu saldo é R$ {}."

# 2. Inserindo os valores com o .format()
frase_final = modelo_frase.format("Sergio", 100)

print(frase_final)
# Saída: Olá, Sergio! Seu saldo é R$ 100.
```

Olá, Sergio! Seu saldo é R\$ 100.

³A palavra placeholder significa “marcador de lugar”. No Python, os placeholders mais comuns são as chaves {}. Elas servem para avisar ao interpretador: “Guarde esse espaço aqui, porque um dado vai entrar neste local mais tarde”.

Os placeholders também podem se referir às variáveis diversas vezes.

```
# Usando variáveis diversas vezes pelo índice
s = """Black Knight: 'Tis but a {0}.
King Arthur: A {0}? Your arm's off!
"""
print(s.format("scratch"))
```

```
Black Knight: 'Tis but a scratch.
King Arthur: A scratch? Your arm's off!
```

Quando você define um índice (como {0}), você está dizendo ao Python: “Busque sempre o primeiro argumento que foi passado dentro do .format()”. Isso permite que você replique o mesmo valor ao longo do texto sem precisar digitá-lo várias vezes no método.

Se você tivesse passado mais palavras, poderia alternar entre elas usando {0}, {1}, {2}, etc.

```
# {0} será o Nome e {1} será a Profissão
frase = """Porteiro: Quem é você?
Visitante: Meu nome é {0}.
Porteiro: Ah, você é o {1}?
Visitante: Sim, eu mesmo, o {1} {0}!
"""

print(frase.format("Sergio", "Programador"))
```

```
Porteiro: Quem é você?
Visitante: Meu nome é Sergio.
Porteiro: Ah, você é o Programador?
Visitante: Sim, eu mesmo, o Programador Sergio!
```

Também podemos fornecer uma variável aos placeholders.

```
s = 'Hayden Planetarium Coordinates: {lat}, {lon}'  
print(s.format(lat='40.7815° N',\  
    lon='73.9733° W'))
```

Hayden Planetarium Coordinates: 40.7815° N, 73.9733° W

3.5.3 Formatação de números

Os números também podem ser formatados.

```
print('Some digits of pi: {}'.\
      format(3.14159265359))
```

Some digits of pi: 3.14159265359

É possível até mesmo formatar números e usar vírgulas como separadores de milhar.

```
print("In 2005, Lu Chao of China recited: {:,} digits of pi.".\
      format(67890))
```

In 2005, Lu Chao of China recited: 67,890 digits of pi.

Números podem ser usados para fazer um cálculo e podem ser formatados com determinado número de valores decimais.

A seguir, calcularemos uma proporção e formataremos esse valor com um percentual.

```
# 0 0 em (0:.4) e em (0:.4%) refere-se ao índice 0 nesse formato;
# 0 .4 refere-se à quantidade de valores decimais, 4;
# Se especificarmos um %, o decimal será formatado como uma porcentagem.

print("I remenber {0:.4} or {0:.4%} of what Lu Chao recited.".\
      format(7/67890))
```

I remenber 0.0001031 or 0.0103% of what Lu Chao recited.

Por fim, podemos usar a formatação de strings para preencher um número com zeros, semelhante ao modo como `zfill` funciona com strings. Ao lidar com dados, esse método pode ser útil se estivermos trabalhando com números de ID lidos como números, mas que devem ser strings.

```
# o primeiro 0 refere-se ao índice nesse formato;  
# o segundo zero refere-se ao caractere para preencher;  
# o 5 nesse caso refere-se à quantidade de caracteres no total;  
# d indica que um dígito será usado;  
  
# o número é preenchido com 0s de modo que a string como um todo tem  
# 5 caracteres.  
  
print("My ID number is {0:05d}".format(42))
```

My ID number is 00042

3.5.4 Formatação no estilo do printf de C

Outra maneira de efetuar uma formatação de strings em Python é por meio do operador `%`. Essa opção segue o estilo de formatação do `printf` de C.

De acordo com a documentação, o método `str.format` apresentado na seção 8.5.3 é preferível. Apesar disso, você ainda poderá ver exemplos de código que usam esse estilo de formatação.

Não daremos muitos detalhes sobre esse método, mas eis alguns dos exemplos anteriores recriados com o estilo de formatação do `printf` do C.

```
# d representa um dígito inteiro

s = 'I only know %d digits of pi' % 7
print(s)
```

I only know 7 digits of pi

```
# s representa uma string;
# observe que o padrão da string usa parênteses ()
# em vez de chaves {};
# a variável passada é um dicionário Python, que usa {}.
print('Some digits of %(cont)s: %(value).2f' %\
      {'cont': 'e', 'value': 2.718})
```

Some digits of e: 2.72

3.5.5 Strings literais formatadas em Python 3.6+ - f-strings

Strings literais formatadas, f-strings, são um recurso novo em Python. A sintaxe é bem parecida com aquela usada na seção “Formatação de strings personalizada” (3.5.1). A principal diferença é que nossa string deve começar com a letra f. Essa sintaxe diz a Python que temos uma string literal formatada. Podemos então usar a variável diretamente no placeholder, {}, sem chamar format.

```
var = 'flesh wound'
s = f"It's just a {var}!"
print(s)
```

It's just a flesh wound!

```
lat = '40.7815° N'
lon = '73.9733° W'
s = f'Hayden Planetarium Coordinates: {lat}, {lon}'
print(s)
```

Hayden Planetarium Coordinates: 40.7815° N, 73.9733° W

As principais vantagens de usar f-strings é que elas são mais legíveis, podem ser mais rápidas e oferecem melhor desempenho.

3.6 Expressões regulares (Regex)

Quando os métodos básicos de string de Python que procuram padrões não forem suficientes, você pode apelar para as expressões regulares a fim de resolver o problema.

As expressões regulares - que são extremamente eficazes - possibilitam uma forma (não trivial) de encontrar e fazer a correspondência de padrões em strings. A desvantagem é que, depois de terminar de escrever uma expressão regular complexa, será difícil descobrir o que o padrão faz somente olhando para ele, ou seja, a sintaxe é difícil de ler.

Em muitas tarefas com dados, como fazer a correspondência com um número de telefone ou efetuar uma validação do campo endereço, quase sempre é mais fácil pesquisar no Google a qual tipo de padrão você está querendo corresponder, e colar o código que alguém já escreveu em seu próprio código (não se esqueça de documentar o local do qual você obteve o padrão).

3.6.1 Correspondência de padrão

3.6.2 Encontrando um padrão

3.6.3 Substituindo um padrão

3.6.4 Compilando um padrão

3.7 Biblioteca `regex`

3.8 Conclusão

4 Apply

5 Operações groupby: separar-aplicar-combinar